Mac 桌面清理工具 技术方案（第一版）

> 产品名未定，暂用"Mac 桌面清理工具"。本文档基于《Mac 桌面清理工具 需求文档（第一版）》。

结论

正式产品用方案一（原生 **Mac** 应用）。 如果想先确认"自己真的会每天用"，可以先用方案三搭一个简易版试用一周，再做方案一。方案二不推荐。

	方案一：原生 **Mac** 应用	方案二：网页包装	方案三：脚本验证
能不能卖	能	能	不能
对零基础的友好程度	中	低	高
应用本身占不占资源	很轻	偏重	很轻
做到能用的工作量	中	大	很小
推荐	正式产品用这个	不推荐	可选，先验证想法

方案一（推荐）：原生 Mac 应用

用大白话说

把这个应用想象成一个住在屏幕右上角菜单栏里的保洁管家：

- 你喊一声（点按钮或按快捷键），它就把桌上不用的东西收进抽屉（隐藏应用），而不是扔掉（关闭应用）。
- 收之前它会拍张照，记下哪些东西是它收的。你说"恢复"，它就按照片放回去。
- 它手里有几张名单（场景），名单上的东西它碰都不碰。
- 看到很久没人用的东西，它会问你"这个要扔吗？"，你签字同意才会扔（退出应用）。

技术选型

用什么	是什么	为什么选它
Swift	苹果官方的编程语言	做 Mac 应用的"正统"语言，AI 写得很熟练

用什么	是什么	为什么选它
SwiftUI	苹果官方的界面工具	用很少的代码就能画出菜单栏面板、按钮、列表
AppKit	苹果官方的系统功能库	列出正在运行的应用、隐藏、退出都靠它
Xcode	苹果官方的开发软件，免费	把代码变成能双击打开的应用
KeyboardShortcuts（开源库）	现成的快捷键工具	全局快捷键自己写很麻烦，用现成的

关键好处：隐藏应用、退出应用都不需要用户在系统设置里额外授权，装好就能用，符合"零学习成本"。

数据流转

一键清理：

1. 用户点"一键清理"或按快捷键。
2. 管家查看现在开着哪些应用。
3. 逐个判断：是用户正在用的吗？在当前场景名单上吗？
   - 是：不动。
   - 否：隐藏，并记到"清理记录"里。
4. 弹出提示"已收起 N 个应用"，播放一声提示音。
5. 如果有很久没用的应用，弹出勾选框，问用户要不要退出。
6. 用户勾选确认后，礼貌地请这些应用退出。应用如果有没保存的内容，会自己弹窗问用户。

一键恢复： 照着"清理记录"，把管家收起来的应用逐个放回来。

快速找回： 菜单里列出"清理记录"中还处于隐藏状态的应用，点一下就叫回来。

数据结构草图

即这个应用需要记住哪些信息：

```
场景
  ├─ 名字：比如 "AI 工作"
  └─ 保留的应用：[ChatGPT, Claude, 飞书, Chrome]
       （用应用的"身份证号"记录，不用名字）

清理记录（每次清理时拍的照）
```

```
   ├─ 清理时间
   ├─ 当时正在用的应用
   └─ 被收起的应用：[微信，网易云音乐，访达 ...]

应用使用记录（用来判断"很久没用"）
   └─ 每个应用最后一次被用的时间：飞书 → 10:32，微信 → 昨天 18:05

设置
   ├─ 当前场景
   ├─ 提示音：开 / 关
   └─ 快捷键：⌥⌘K
```

"身份证号"指每个 Mac 应用唯一的编号（技术上叫 Bundle ID）。它不会因为系统语言是中文还是英文而变化。

优缺点和取舍

- 优点： 运行最轻快，工具本身不会让电脑变卡（"防卡工具"自己不能卡）；外观和系统统一，看起来正规；以后想加功能（比如记录窗口位置）有现成的路。
- 缺点： 只能做 Mac 版；必须有一台 Mac 并安装 Xcode。想卖给别人，需要加入苹果开发者计划（每年 99 美元），否则别人打开时会被系统拦住。
- 取舍： 需求本来就只做 Mac，所以"只能做 Mac 版"几乎不算缺点。

方案二（不推荐）：用网页技术包装成桌面应用

用大白话说

就像在电脑上开了一个专门的小浏览器。界面其实是一个网页，再找一个"翻译"去和 Mac 系统打交道，让它帮忙隐藏、退出应用。

技术选型

- **Tauri** 或 **Electron**：把网页打包成桌面应用的工具。
- 网页技术（HTML、CSS、JavaScript）：画界面。
- 隐藏、退出应用的部分，仍然要借助苹果的系统接口，需要另外写一段 Swift 或 Rust 代码当"翻译"。

数据流转

网页界面上点按钮 → 翻译层 → 苹果系统接口 → 隐藏或退出应用 → 结果传回网页显示。

数据结构

与方案一相同。

优缺点和取舍

- 优点： 网上做网页的教程和素材最多，界面容易做得花哨；以后理论上能做 Windows 版。
- 缺点： 核心功能绕不开苹果的接口，等于要学两套东西；中间多一层翻译，出问题时很难判断错在哪；用 Electron 的话应用本身更占内存，和"防卡"的卖点矛盾。
- 取舍： 为了一个暂时用不上的"跨平台"，换来了更高的复杂度。

方案三（可选）：用现成工具写脚本，先验证想法

用大白话说

先不盖房子，搭个帐篷试住几天。用一个叫 Hammerspoon 的免费工具写几十行脚本，就能做到"按快捷键，除了名单上的应用全部隐藏"。

技术选型

- **Hammerspoon**：Mac 上的免费自动化工具。
- **Lua**：它使用的脚本语言，很简单，AI 可以帮你写。

数据流转

按快捷键 → 脚本列出所有应用 → 不在名单上的就隐藏 → 屏幕角落弹出一行提示。

数据结构

名单直接写在脚本里，例如：`保留 = {"ChatGPT", "飞书", "Google Chrome"}`。

优缺点和取舍

- 优点： 很快就能用上，几乎零成本验证一件事：清理之后，自己是不是真的觉得舒服、觉得顺了。
- 缺点： 没有像样的界面；别人要先装 Hammerspoon 才能用；不能拿来卖。
- 取舍： 它不是产品，而是验证工具。

需要准备的东西

- 一台 Mac，从 App Store 免费安装 **Xcode**。
- 准备卖给别人时，加入苹果开发者计划（每年 99 美元），用于签名和公证。

以后自己改代码时容易踩的坑（以方案一为准）

1. "当前正在用的应用"会被认错（最常见）。 点菜单栏图标的那一刻，"当前应用"就变成了管家自己。如果代码直接问系统"现在谁在前台"，用户真正在用的应用会被收掉。正确做法：管家要随时记住"上一个被使用的应用"。

2. 用应用名字认人会认错。 飞书在中文系统里叫"飞书"，在英文系统里可能叫"Feishu"或"Lark"。名单里永远存应用的"身份证号"，界面上再显示名字。
3. "恢复"时放回了不该放回的东西。 用户在清理前自己手动隐藏的应用，恢复时不应该弹出来。只恢复"清理记录"里由管家亲手收起来的应用。
4. "很久没用"的记录会丢。 系统不会告诉你某个应用上次是什么时候用的，只能靠管家运行期间自己记。使用记录要存到硬盘上；刚装好的前几个小时，这个功能天然不准。
5. 退出应用只能"礼貌地请"，不能"强行关"。 强制退出会直接丢掉没保存的内容。AI 有时为了"保证退出成功"会改用强制退出。原则写死：永远不用强制退出。
6. 改了"记住的信息"的格式，旧数据会读不出来。 比如给"场景"加了一个新字段，用户以前保存的场景可能全部丢失。每次修改格式时，要让 AI 同时处理旧格式的兼容。
7. 自己电脑能打开，不代表别人电脑能打开。 没做苹果的签名和公证，别人打开时会看到"已损坏"或"无法验证开发者"。这不是代码问题，给别人用之前要专门做一次签名和公证。

给零基础开发者的建议

- 每次只让 **AI** 改一件事，改完马上试用，没问题再改下一件。
- 每改好一次就用 git 存一个版本（相当于游戏存档点）。改坏了，直接退回上一个存档。
- 每次让 AI 改代码时，把需求文档里的"原则"和本文档里的"容易踩的坑"一起发给它。